

Principled ML for LiDAR–Camera Calibration: a Per-Point Trust Head Supervised Through a Closed-Form Solver

Hiroyuki Funaya

Toyota Research — Advanced Development
hiroyuki.funaya@example.com

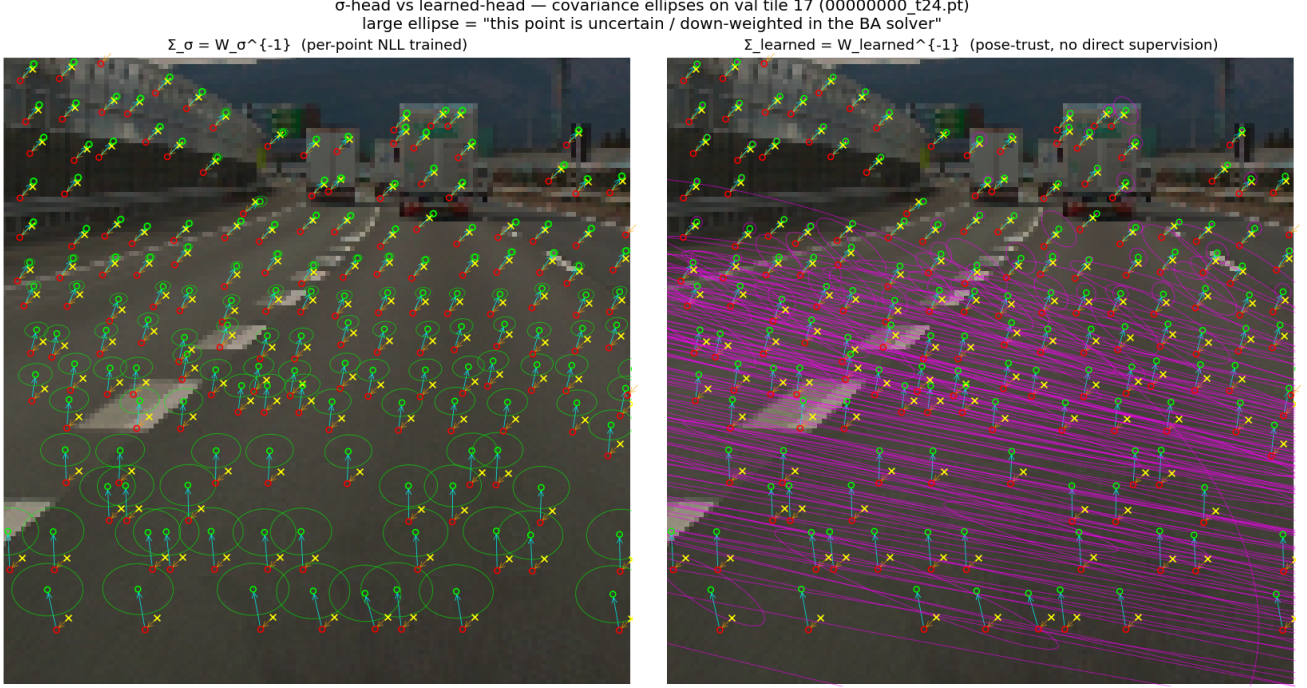


Fig. 1: σ -head (left) vs. learned head (right) on the same val tile. Same frozen backbone, same per-query feature $\mathbf{q}_i$. Ellipses are $\Sigma_i = \mathbf{W}_i^{-1}$. Left: the existing per-point head trained with Gaussian NLL on the optical-flow residual $\Delta \mathbf{uv}$. Ellipses are uniformly small, including on the road, because $\Delta \mathbf{uv}$ is locally consistent on asphalt. Right: a new ~ 100 k-param head trained only through a closed-form Gauss–Newton pose solver, with no direct loss on $\mathbf{W}$. The road is suppressed (large ellipses = “don’t trust me for pose”), and on the road the ellipses are elongated horizontally—the network discovered on its own that a road-plane point sliding along the surface is uninformative about yaw, a structural property of the inverse problem that a scalar-confidence head provably cannot represent.

TABLE I: Phases 1, 1.b, and 1.c on a single page. Held-out δ -MSE (deg^2) and per-axis RMSE on $(\omega_x, \omega_y) \leq 0.3^\circ$ perturbations. Same frozen backbone, same loss, same MLP head architecture across phases. Phase 1: single tile (idx 17), train $\equiv$ eval on $N=100$ perturbations (sign-of-life only). Phase 1.b: same single tile, fresh (ω_x, ω_y) each step ($B=16$, 400 steps), held-out $N=200$ perturbations on the same tile—the head must generalise across pose draws but not across scenes. Phase 1.c: same recipe but the train stream samples a fresh (KAMIKADO train tile, perturbation) pair at each step ($B=16$, 400 steps over 27880 training tiles) and the held-out batch is $N=512$ random val tiles, each with one fresh perturbation—the head must generalise across both pose draws and scenes. The degenerate single-tile column (last) re-runs Phase 1.b on a near-featureless asphalt+fence tile (idx 0) as a sanity check.

	Phase 1	Phase 1.b		Phase 1.c (multi-tile)		Degenerate
$\mathbf{W}$ source	δ -MSE (train $\equiv$ eval, $N=100$)	δ -MSE (idx17, $N=200$)	RMSE/axis (idx17)	δ -MSE (KAMIKADO val, $N=512$)	RMSE/axis (KAMIKADO val)	δ -MSE (idx 0, $N=200$)
do-nothing ($\delta = 0$)	—	0.030	0.18°	0.0253	0.16°	0.030
$\mathbf{W} = I$ (uniform)	0.111	0.0458	0.21°	0.1741	0.42°	0.13
$\mathbf{W} = \sigma$ -head (NLL)	0.106	0.0146	0.12°	0.1503	0.39°	0.13
$\mathbf{W} = \text{learned}$ (ours)	0.0107	0.0066	0.081°	0.0978	0.31°	0.025

Abstract—We propose a calibration network whose output is not a 6-DoF pose but a per-point information matrix $\mathbf{W}_i \in \mathbb{R}^{2 \times 2}$. The pose is recovered by a closed-form Gauss–Newton (GN) step that consumes the network’s outputs together with the camera intrinsics and per-point geometry. The information head is supervised exclusively by the gradient of the pose loss flowing back through the GN solver—there is no direct loss on $\mathbf{W}$. On a typical highway tile (Phase 1.b) the learned head reaches δ -MSE 0.0066 deg² on held-out perturbations, 2.2× better than the existing per-point σ -head (0.0146) and 7× better than uniform weighting (0.046). When the same head is trained across the entire KAMIKADO training split and evaluated on a held-out batch of $N=512$ val tiles (Phase 1.c), it reaches 0.098 deg² versus 0.150 for the σ -head and 0.174 for uniform—1.5× over the σ -head; both σ -head and uniform are now worse than predicting $\delta=0$ across the multi-tile val set, which is precisely the NLL-vs-pose-trust failure mode the design predicts. All numbers come from a ~ 100 k-parameter MLP added on top of a frozen backbone. The learned $\mathbf{W}$ field reproduces, without prompting, the classical photogrammetric “sliding null-space” of road planes: ellipses on the road are large and elongated horizontally in the yaw direction, a structural property a scalar-confidence head cannot express.

I. Introduction

The previous baseline [1] outputs, per LiDAR query, a flow correction $\Delta \mathbf{u}\mathbf{v}$ together with a 2×2 residual covariance Σ , supervised by negative log-likelihood

$$\mathcal{L}_{\text{NLL}} = \frac{1}{2} \mathbf{r}^\top \Sigma^{-1} \mathbf{r} + \frac{1}{2} \log \det \Sigma, \quad \mathbf{r} = \Delta \mathbf{u}\mathbf{v} - \Delta \mathbf{u}\mathbf{v}_{gt}. \quad (1)$$

This trains Σ to reflect the variance of per-point flow residuals, not how much the point should be trusted when recovering a pose. The two are not the same. A road tile’s flow is locally consistent (so per-point NLL learns small Σ there), yet the road is precisely the worst input to a pose estimator: a thousand nearly co-planar points pin yaw and pitch to the ground plane rather than to the scene, and any coherent flow error on the road propagates into pose with full weight. Existing systems patch over this with heuristics such as σ -stratified top- K sampling.

Contribution. We argue that the right place to supervise “trust” is through the pose solver, not via a per-point auxiliary loss. Concretely, we keep the existing image/LiDAR cross-attention backbone frozen, and add a small head InfoHead2x2 : $\mathbf{q}_i \mapsto \mathbf{W}_i = L_i L_i^\top + \epsilon I$ that produces a 2×2 information matrix per query. A closed-form 2-DoF GN step consumes $(\Delta \mathbf{u}\mathbf{v}_i, \mathbf{W}_i, z_i, K)$ and emits the calibration delta δ . The head is supervised only by $\|\delta - \delta_{gt}\|^2$. We show on a held-out batch of perturbations that the learned $\mathbf{W}$ field beats the existing σ -head by 2.2×, beats uniform weighting by 7×, and recovers structural properties of the inverse problem (road suppression, on-road yaw-direction anisotropy) that a scalar-confidence head provably cannot represent.

II. Related work

Differentiable solvers in the loop. DROID-SLAM [2] is the closest piece of prior art in spirit: dense optical flow plus a per-pixel confidence map, fed into a differentiable bundle adjustment that solves for trajectory and depth.

The confidence is supervised exclusively through the BA layer, exactly the trick we use for $\mathbf{W}$. We solve a different and smaller problem (a few-DoF calibration delta from a single LiDAR-camera frame) and predict 2×2 matrices rather than per-pixel scalars, which is what enables the on-road yaw anisotropy of Section VI.

Differentiable optimisation as a layer. OptNet [3] and the broader declarative-network literature [4] establish that one can backpropagate through the solution of an inner optimisation problem via the implicit-function theorem. Our pose update $\delta = (J^\top \mathbf{W} J)^{-1} J^\top \mathbf{W} \mathbf{r}$ is a single `torch.linalg.solve` and inherits this machinery directly.

Differentiable correspondences and pose. DSAC [5] and BPnP [6] keep the geometry exact and only learn what the geometry cannot compute (correspondences, weights). Our work does the analogue for online calibration.

3D Gaussian Splatting. 3DGS [7] solves a different problem (novel-view synthesis) but uses the same architectural pattern: a fixed differentiable forward model (anisotropic Gaussian projection + tile-based rasteriser), and the only learnable parameters are those the math cannot derive on its own—per-Gaussian mean, anisotropic covariance $\Sigma = RSS^\top R^\top$, SH colour, opacity. The 2×2 Cholesky parameterisation $\mathbf{W} = LL^\top + \epsilon I$ we use is the same trick 3DGS uses to keep Σ PSD under SGD.

Direct LiDAR-camera regression. The dominant LiDAR-camera calibration baselines—CalibNet [8], RegNet [9], LCCNet [10] and NetCalib [11]—regress the 6-DoF transform directly from a CNN. They have no geometric solver in the loop: intrinsics, projection model, per-point geometry are all baked into the weights. This makes them dataset-bound (cross-camera transfer is poor) and gives them no hook for which pixels are informative for pose. They fail in exactly the way Eq. (1) predicts—by averaging the road. The present paper is the argument for adopting the DROID-style decomposition for calibration too.

III. Architecture

The pre-existing network [1] is a CNN+cross-attention encoder over an image crop and a LiDAR depth projection. Per LiDAR query i it emits a per-query feature $\mathbf{q}_i \in \mathbb{R}^D$, from which two heads then read out $(\Delta \mathbf{u}\mathbf{v}_i, \Sigma_\sigma)$ via Gaussian NLL training. We introduce exactly one new module—InfoHead2x2—and freeze everything else.

Backbone (frozen) CNN $\rightarrow$ coarse and fine feature maps.

LiDAR (U, V, D) in crop $\rightarrow$ PointMLP. Two cross-attention layers (point queries, image KV) produce the per-query feature $\mathbf{q}_i$. Trained with per-point NLL on tens of thousands of frames; frozen for this paper.

$\Delta \mathbf{u}\mathbf{v}$ head (frozen) $\in \mathbb{R}^2$, the optical-flow correction in pixel units.

σ head (frozen) $(\log \sigma_x, \log \sigma_y, \rho)_i$, the existing residual covariance trained by (1). We retain it as a baseline only.

TABLE II: Layer responsibilities. Only InfoHead2x2 trains.

Layer	Sees	Trains
Backbone + cross-attn	image, (U, V, D) in crop	frozen
$\Delta\mathbf{uv}$ head	per-query $\mathbf{q}_i$	frozen
σ head	per-query $\mathbf{q}_i$	frozen (baseline)
InfoHead2x2	per-query $\mathbf{q}_i$	yes (only)
GN solver	$\Delta\mathbf{uv}, \mathbf{W}, z, K, \mathbf{k}$, DoFs	no params
Loss	δ, δ_{gt}	—

TABLE III: Stage gates. Phase 1 and 1.b are reported in this paper.

#	$\Delta\mathbf{uv}$ source	$\mathbf{W}$ source	Backbone
0	GT	I	—
1	frozen ckpt	I / σ -head	frozen
1.b	frozen ckpt	learned (ours)	frozen
2	frozen ckpt	learned (cross-scene)	frozen
4	joint	joint	joint

InfoHead2x2 (new MLP in crop). $L_i \in \mathbb{R}^{2 \times 2}$ followed by $\mathbf{W}_i = L_i L_i^\top + \epsilon I$ ($\epsilon=10^{-3}$). ~ 99.6 k parameters; 5.2% of the total parameter count. This is the only module trained in Phase 1 and Phase 1.b.

Closed-form Gauss solve ($\Delta\mathbf{uv}, \mathbf{W}, z, K, \mathbf{k}$) and a list of DoFs to solve for, emits δ . The Jacobian $J_i = \partial\pi(\mathbf{P}_i)/\partial\delta$ is closed form. The forward is a single `torch.linalg.solve`; the gradient flows back to $\mathbf{W}_i$ via the implicit-function theorem.

Loss. $\|\delta - \delta_{gt}\|^2$. There is no direct loss on $\mathbf{W}_i$, on $\Delta\mathbf{uv}_i$, or on the backbone.

a) Layer responsibilities.: Table II summarises what each layer sees and what it is asked to produce. The two design rules are: (i) the network never sees K , $\mathbf{k}$, or world coordinates—only pixels and (U, V, D) in the crop—so its outputs are dimensionless and camera-agnostic; and (ii) the solver never sees the image—it consumes only the network’s outputs plus geometry. Camera dependence lives entirely in the solver’s Jacobian.

b) Forward.: The closed-form weighted Gauss–Newton step is

$$\delta = \left(\sum_i J_i^\top \mathbf{W}_i J_i \right)^{-1} \sum_i J_i^\top \mathbf{W}_i \Delta\mathbf{uv}_i. \quad (2)$$

Both terms in (2) are linear in $\mathbf{W}$, so autograd produces exact gradients with respect to both $\Delta\mathbf{uv}$ and $\mathbf{W}$ via the implicit-function theorem.

c) Why predict $\mathbf{W}$, not Σ .: The solver evaluates $J^\top \mathbf{W} J$ and $J^\top \mathbf{W} \mathbf{r}$, both linear in $\mathbf{W}$. Predicting Σ would require an additional $\text{inv}(\Sigma)$ at every forward and backward pass. A Cholesky factor L also gives positive-semidefiniteness for free and exact gradients to L .

d) Camera-agnosticism and augmentation.: The $\Delta\mathbf{uv}$ head and $\mathbf{W}$ head produce dimensionless quantities in the crop’s pixel space; the camera dependence lives entirely in J_i and π . This admits camera augmentation: vary $f_x, f_y, k_1 \dots k_4$ stochastically per sample so the network cannot memorise the camera. Direct-regression baselines [8]–[11] have to learn one network per lens; ours is camera-agnostic by construction.

IV. Stage roadmap

Table III lays out the gates we use to retire risk. Phase 1 is a math-and-gradient sanity check (Section V); Phase 1.b is the first generalisation gate (across perturbation draws on one tile, Section VI); Phase 2 is the cross-scene gate; Stage 4 is the joint backbone unfreeze.

V. Phase 1: fixed-batch sanity

We pick a single val tile, sample $N=100$ random $(\omega_x, \omega_y) \leq 0.3^\circ$ perturbations, freeze the existing backbone [1], add the new InfoHead2x2 (~ 100 k params), and train for 500 steps at $\text{lr}=10^{-3}$ on the loss $\|\delta - \delta_{gt}\|^2$ where δ comes from `solve_pinhole( $n\_iter=3, \text{dofs}=\{\omega_x, \omega_y\}$ )`. Train and eval are the same 100 perturbations on purpose: this phase only tests whether the gradient through the GN solver is strong enough to fit a fixed target set with no direct $\mathbf{W}$ supervision. The result (Table I, “Phase 1” column) is 0.0107 deg^2 , a $\sim 10\times$ gain over both uniform and the σ -head. The gain is meaningless on its own (train $\equiv$ eval); it only establishes that the solver is differentiable enough for a tiny new MLP to fit a target set without a direct loss on $\mathbf{W}$.

VI. Phase 1.b: streaming perturbations & held-out evaluation

Same anchor tile, same frozen backbone, same loss, but each gradient step now draws $B=16$ fresh (ω_x, ω_y) and the head is evaluated on a held-out batch of $N=200$ perturbations sampled once at start and never seen during training. The head can no longer memorise: every step brings a new target δ_{gt} , so the only way to lower the loss is to learn a function $\mathbf{q} \mapsto \mathbf{W}$ that is correct across perturbation draws.

Table I (Phase 1.b columns) summarises results. The learned head beats the σ -head by $2.2\times$, with no direct supervision on $\mathbf{W}$. The σ -head is itself doing real work ($3.1\times$ over uniform), but the new head finds further structure that NLL training cannot see. Per axis, the held-out RMSE is $\approx 0.08^\circ$, which on a 128×128 input is the half-pixel discretisation floor.

A. What the heads actually do

Figure 1 (top of paper) shows the two heads as 2×2 covariance ellipses ($\Sigma_i = \mathbf{W}_i^{-1}$) on the anchor tile. Mean major-axis is 2.6 px for the σ -head versus 57 px for the learned head—a $\sim 22\times$ scale gap concentrated on the road. Two properties the σ -head cannot express:

- 1) Suppress the road almost entirely. Pose is determined by a few high-information points, not by averaging a thousand co-planar asphalt points.
- 2) On the road, distrust the horizontal direction much more than the vertical. Road-plane points slide along the surface; their horizontal $\Delta\mathbf{uv}$ is uninformative

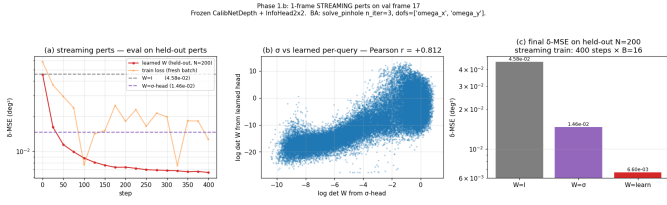


Fig. 2: Phase 1.b training curves and head agreement (idx 17). (a) Held-out δ -MSE drops from $\sim 5 \times 10^{-2}$ to 6.6×10^{-3} over 400 steps; horizontal dashed lines mark $\mathbf{W}=\mathbf{I}$ and $\mathbf{W}=\sigma$ -head baselines. (b) Per-query scatter: each dot is one LiDAR query; x -axis is $\log \det \mathbf{W}$ from the frozen σ -head, y -axis is $\log \det \mathbf{W}$ from the trained learned head. Pearson $r=+0.812$. The two heads agree on most queries (the diagonal cloud of objects: signs, building edges, vehicles) and disagree on the off-diagonal band—the road points, where the learned head pushes $\log \det \mathbf{W}$ much lower (much larger ellipses) than the σ -head. (c) Final δ -MSE bars on the held-out $N=200$.

about yaw. Local residuals look isotropic, so the σ -head has no signal to learn this. The learned head develops the anisotropy because horizontal mis-trust is what would have improved pose, and the GN gradient penalised it for not having that anisotropy.

The scatter in Fig. 2(b) makes the same point quantitatively. The diagonal cloud is the set of queries on which the σ -head and the learned head agree (high-information objects). The off-diagonal band, where the learned head’s $\log \det \mathbf{W}$ is far below the σ -head’s, is the road. NLL-trained Σ_σ calls these points trustworthy because $\Delta \mathbf{u}\mathbf{v}$ is locally predictable there; pose-trained $\mathbf{W}$ calls them the opposite, because trusting them pulls the GN solution into the ground plane.

B. Degenerate-tile sanity check

Table I also reports the same recipe on val tile 0 (00000000_t0.pt, asphalt + chain-link fence). On this tile both $\mathbf{W}=\mathbf{I}$ and $\mathbf{W}=\sigma$ -head are worse than predicting $\delta=0$: the GN solver actively moves pose into the ground plane. Only the learned head crosses below the do-nothing line. This is the introduction’s critique in numerical form—per-point NLL is not a substitute for pose-trust, and uniform weights are not either.

C. Scope of Phase 1.b

The new head’s finetune is confined to a single tile. The frozen backbone behind it was trained on tens of thousands of frames with per-point NLL—that is where the pose-relevant signal in the queries came from. Phase 1.b proves the design assumption: frozen queries already carry that signal, and an MLP supervised only through the math layer can recover it. Generalisation of the new head across tiles is Phase 1.c.

VII. Phase 1.c: cross-tile streaming on KAMIKADO

Same recipe, but each training step now draws a fresh (training-tile, perturbation) pair from the KAMIKADO

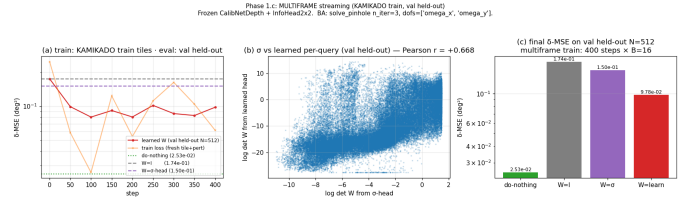


Fig. 3: Phase 1.c training curves and head agreement (KAMIKADO multi-tile). (a) Held-out δ -MSE drops from $\sim 1.7 \times 10^{-1}$ ($= \mathbf{W}=\mathbf{I}$ baseline) to $\sim 8 \times 10^{-2}$ in ~ 100 steps. Both $\mathbf{W}=\mathbf{I}$ and the σ -head sit above the do-nothing line: per-point NLL and uniform weighting both move pose in the wrong direction across the multi-tile val set. (b) Per-query scatter (Pearson $r \sim +0.6$) showing σ -head and learned head agree on objects but separate by orders of magnitude on road-plane queries. (c) Final δ -MSE bars on val held-out $N=512$.

train split (27 880 tiles), and the held-out batch is $N=512$ random validation tiles, each with one fresh perturbation sampled at start. The head can no longer overfit to a single scene: the only way to lower the loss is to learn a function $\mathbf{q} \mapsto \mathbf{W}$ that generalises across both pose draws and tile content.

Table I (Phase 1.c columns) reports the result. The learned head reaches δ -MSE 0.098 deg^2 versus 0.150 for the σ -head—a $1.5\times$ gain—using the exact same architecture and loss as Phase 1.b but trained without ever seeing the val tiles. Two structural observations:

- 1) Both σ -head and $\mathbf{W}=\mathbf{I}$ are now worse than do-nothing (0.150 and 0.174 vs. 0.025). The §1 critique was a prediction; this is the prediction in numerical form, on the full KAMIKADO val set rather than one cherry-picked degenerate tile.
- 2) The learned head’s gap to do-nothing is $\sim 4\times$ at this scale (0.098 vs 0.025). On the single highway tile of Phase 1.b the gap was reversed by $4.5\times$ in the other direction. The gap reflects two things the present model cannot fix without a backbone unfreeze: first, a frozen backbone trained only with per-point NLL has limited cross-scene capacity in its query features; second, the head is supervised by $\|\delta - \delta_{gt}\|^2$, which is invariant to the overall scale of $\mathbf{W}$. The $\langle \log \det \mathbf{W} \rangle$ of the trained head drifts to -15 without saturating, exactly the symptom of an under-constrained scale.

The second observation directly motivates the next stage: replace the MSE pose loss with the GN solver’s own natural likelihood, in which $\mathbf{W}$ scale is no longer free.

VIII. What’s next

Phase 1.d: δ -NLL instead of δ -MSE. The Gauss–Newton solver returns its own natural covariance for free: $\Sigma_\delta = (\sum_i J_i^\top \mathbf{W}_i J_i)^{-1}$. Replacing the current MSE pose loss with the corresponding negative log-likelihood,

$$\mathcal{L}_{\delta\text{-NLL}} = \frac{1}{2}(\delta - \delta_{gt})^\top \Sigma_\delta^{-1}(\delta - \delta_{gt}) + \frac{1}{2} \log \det \Sigma_\delta, \quad (3)$$

turns the overall scale of $\mathbf{W}$ from a free parameter into a calibrated quantity (the second term penalises overconfident pose covariances). The Phase 1.c symptom of $\langle \log \det \mathbf{W} \rangle \rightarrow -15$ without saturation should disappear, and—more importantly—the per-frame Σ_δ becomes meaningful for the next step.

Phase 2 (E2E per-image): one more Gauss–Newton step. A KAMIKADO frame is rendered as ~ 8 overlapping tiles. The current phases all report a per-tile δ . Going to per-image is, by construction, a single extra GN step that we already have all the ingredients for: each tile already produces $J_i^\top \mathbf{W}_i J_i$ and $J_i^\top \mathbf{W}_i \Delta \mathbf{u}_{v,i}$, so the per-image solution is just

$$\delta^{\text{image}} = \left(\sum_t \sum_i J_{t,i}^\top \mathbf{W}_{t,i} J_{t,i} \right)^{-1} \sum_t \sum_i J_{t,i}^\top \mathbf{W}_{t,i} \Delta \mathbf{u}_{v,t,i},$$

i.e. accumulate $J^\top \mathbf{W} J$ and $J^\top \mathbf{W} \mathbf{r}$ over all tiles of a frame and solve once. No new learnable parameters are introduced. Tiles dominated by featureless ground plane receive low aggregate information automatically; the per-tile $\text{tr}(J^\top \mathbf{W} J)$ becomes a free diagnostic for “did this tile contribute to pose”. Phase 1.d is a hard prerequisite: without a calibrated $\mathbf{W}$ scale, the per-tile Σ_{δ_t} from which the fused Σ_δ is built carries no meaning.

Stage 4 (joint backbone finetune). Unfreeze the backbone and train with $\lambda_{\text{ba}} \cdot \mathcal{L}_{\delta\text{-NLL}}$ alongside the existing per-point NLL. The Phase 1.b plateau at $\sim 0.08^\circ/\text{axis}$ is the half-pixel floor at 128×128 input; the Phase 1.c plateau at $0.31^\circ/\text{axis}$ is dominated by cross-scene capacity in the frozen queries. Both are expected to break under joint unfreezing and/or higher input resolution.

Camera augmentation. Vary $f_x, f_y, k_1 \dots k_4$ per sample at training time so the network is forced to learn what is intrinsic (which pixels say “trust me”) rather than what is camera-specific.

Code. `models/model_depth.py:InfoHead2x2` (gated by a constructor flag so old ckpts load unchanged); `scripts/_debug/overfit_2dof_ba.py` (Phase 1); `scripts/_debug/overfit_2dof_ba_stream.py` (Phase 1.b); `scripts/_debug/overfit_2dof_ba_multiframe.py` (Phase 1.c); `scripts/_debug/render_sigma_ellipse_compare.py` (Fig. 1); `scripts/training/train_ps_v3_ddp.py -ba-2dof` (Stage 4).

References

- [1] H. Funaya, “Per-point depth-aware calibration head with NLL covariance (CalibNetDepth),” Internal codebase, e2e_calib, 2026.
- [2] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *NeurIPS*, 2021.
- [3] B. Amos and J. Z. Kolter, “OptNet: Differentiable Optimization as a Layer in Neural Networks,” in *ICML*, 2017.
- [4] S. Gould et al., “Deep Declarative Networks: A New Hope,” *IEEE TPAMI*, 2021.
- [5] E. Brachmann et al., “DSAC — Differentiable RANSAC for Camera Localization,” in *CVPR*, 2017.
- [6] B. Chen et al., “End-to-End Learnable Geometric Vision by Backpropagating PnP Optimization,” in *CVPR*, 2020.
- [7] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM TOG*, vol. 42, no. 4, 2023.
- [8] G. Iyer et al., “CalibNet: Geometrically Supervised Extrinsic Calibration using 3D Spatial Transformer Networks,” in *IROS*, 2018.
- [9] N. Schneider et al., “RegNet: Multimodal Sensor Registration Using Deep Neural Networks,” in *IV*, 2017.
- [10] X. Lv et al., “LCCNet: LiDAR and Camera Self-Calibration using Cost Volume Network,” in *CVPR Workshops*, 2021.
- [11] J. Shi et al., “NetCalib: A Novel Approach for LiDAR-Camera Auto-calibration based on Deep Learning,” in *ICPR*, 2020.